

```
=====
Python Control Statements
=====
```

**** Control statements are used to control the flow of execution in a Python program.**

**** Normally, Python executes statements from top to bottom.**

**** But in real-time applications, we need to execute code based on conditions or repeat some code multiple times.**

Ex:

```
if(uname == "admin" and pwd == "admin123"){
    // logic to display dashboard page
}else{
    //logic to display error msg
}
```

Why Do We Need Control Statements?

**** In real-time applications, we need decision-making and repetition.**

Real-Time Examples

- 1) If user logged in, show dashboard page
- 2) If student marks are greater than 35, show pass
- 3) If account balance is sufficient, allow withdrawal / transfer
- 4) Print numbers from 1 to 100
- 5) Send certificates to all students
- 6) Display all products in cart
- 7) Search a student from a list
- 8) Repeat login until correct password is entered

**** Without control statements, programs cannot make decisions or repeat tasks.**

***** Python control statements are mainly divided into three types:**

1. Conditional Statements
2. Looping Statements
3. Jumping Statements (loop control statements)

```
=====
Conditional Statements
=====
```

=> Conditional statements are used to execute the logic based on condition

=> In conditional statements the logic will execute only once.

if statement

** The if statement is used to execute a block of code only when the condition is true.

syntax :

```
if condition:
    statement
```

Example

```
age = 25
```

```
if age>=18:
    print("Eligible to vote")
    print("plz participate in voting")
```

Indentation in Python

** Python uses indentation to define blocks of code.

** Indentation means spaces at the beginning of a line.

wrong syntax

```
age = 20
```

```
if age >= 18:
    print("Eligible to vote")
```

if-else Statement

syntax :

```
if condition:
    statement-1
else:
    statement-2
```

Example

```
age = 12
```

```
if age >= 18:
    print("Eligible to vote")
    print("plz participate in voting")
```

```
else:
    print("Not eligible to vote")
```

if-elif-else statements

** When we want to check multiple conditions like below then we need to use if-elif-else concept.

```
marks >= 90 (A Grade)

marks >= 75 (B grade)

marks >= 60 (C grade)

marks >= 35 (Pass)

else ==> Fail
```

Syntax :

```
if condition-1:
    statement-1
elif condition-2:
    statement-2
elif condition-3:
    statement-3
else:
    default_statement
```

Example

```
marks = int(input("Enter marks: "))

if marks >= 90:
    print("A Grade")
elif marks >= 75:
    print("B Grade")
elif marks >= 65:
    print("C Grade")
elif marks >= 35:
    print("Just Pass")
else:
    print("Failed")
```

Nested If stmt

** Writing one if statement inside another if statement is called as nested if.

Example-1 ::

```
if username == "admin" and pwd == "admin123":

    if role == "student":
        print("student dashboard")

    elif role == "trainer":
        print("trainer dashboard")

    elif role == "admin":
        print("admin dashboard")
else:
    print("Invalid Credentials")
```

Example-2 ::

```
balance = 5000
```

```
withdraw_amount = int(input("Enter amount to withdraw: "))

if balance > 0 :
    if withdraw_amount <= balance:
        balance = balance - withdraw_amount
        print("Withdraw successful")
        print("Remaining balance: ", balance)
    else:
        print("Insufficient balance")
else:
    print("Funds not available")
```

```
##### match case #####
```

```
print("1. Add Student")
print("2. View Student")
print("3. Update Student")
print("4. Delete Student")

choice = int(input("Enter your choice: "))

match choice:
    case 1:
        print("Student add operation selected")
    case 2:
        print("Student View Operation selected")
    case 3:
        print("Student Update Operation selected")
    case 4:
        print("Student Delete Operation selected")
    case _:
        print("Invalid Choice")
```

```
=====
Looping statements
=====
```

**** Looping statements are used to execute a block of code repeatedly.**

**** Python supports two types of loops**

- 1) for loop
- 2) while loop

=> For loop is used to iterate over a sequence

=> Sequence can be

- 1) String
- 2) List
- 3) Tuple
- 4) Set
- 5) Dictionary
- 6) Range

for loop syntax ::

for variable in sequence:

```
statements

### for loop example ::

    for i in range(1,6):
        print(i)

=====
what is range( ) ?
=====

==> The range() function is used to generate a sequence of numbers.

### range( ) function syntax:

    range (stop)

    range (start, stop)

    range (start, stop, step)


## Example ::

# range(stop)
for i in range(6):
    print(i)

# range(start, stop)
for i in range(1, 6):
    print(i)

# range(start, stop, step)
for i in range(1, 11, 2):
    print(i)

# for loop with string
name = "Ashok"
for ch in name:
    print(ch)

# for loop with List
students = ["Ashok", "Ram", "John"]
for student in students:
    print(student)

# for loop with Tuple
courses = ("JAVA", "Python", "AI")
for course in courses:
    print(course)

# for loop with Set
places = {"Hyd", "Banglore", "Pune"}
for place in places:
    print(place)

#for loop with dictionary
student = {
    "id" : 101,
    "name" : "Ashok",
    "gender" : "Male"
}
```

```
for key in student:
    print(key, "--", student[key])

# Cart Price for items
cart_prices = [500, 300, 1200, 750]
total = 0

for price in cart_prices:
    total = total + price

print("Total Cart Price :: ", total)
```

```
=====
While Loop
=====
```

**** The while loop executes a block of code as long as the condition is true.**

while loop syntax ::

```
while condition:
    statements
```

while loop example ::

```
count = 1

while count <= 5:
    print(count)
    count = count + 1
```

```
=====
for loop vs while loop
=====
```

- 1) Both for loop and while loop are used to execute a block of code repeatedly.
- 2) for loop is used when we know how many times the loop should run.
- 3) while loop is used when we do not know exactly how many times the loop will run.
- 4) for loop works with sequences like range, list, tuple, string, etc.
- 5) while loop works based on a condition.
- 6) In for loop, initialization and increment are handled automatically by range() or sequence.
- 7) In while loop, initialization and increment/decrement should be handled manually.
- 8) for loop is mostly used for fixed iterations.
- 9) while loop is mostly used for condition-based iterations.
- 10) for loop has less chance of infinite loop.
- 11) while loop has more chance of infinite loop if the condition is not updated properly.

Requirement : Design an ATM system.

When we run the application it should show Menu Options like below

1. Check Balance
2. Deposit
3. Withdraw
4. Exit

=> Take input from user, based on given input perform the action.

=====
Jumping Statements
=====

**** Jumping statements are used to control the flow of loop execution.**

**** To control loop execution we have**

- 1) break
- 2) continue

what is break statement ?

=> The break statement is used to stop the loop immediately.

**** "Stop the loop right now and come out of it."**

#####

```
for i in range(1, 11):  
    if i == 5:  
        break  
    print(i)
```

#####

```
students = ["Raju", "john", "Steve", "Ashok", "Rani"]  
search_name = "Ashok"
```

```
for student in students:  
    print("Checking :", student)  
    if search_name == student:  
        print("Student Found :", student)  
        break
```

what is continue statement ?

**** The continue statement is used to skip the current iteration and move to the next iteration.**

Note: It does not stop the loop completely.

#####

```
for i in range(1, 11):  
    if i == 5:  
        continue
```

```
    print(i)

#####

students = [
    {"name": "Ravi", "marks": 80},
    {"name": "Kiran", "marks": 30},
    {"name": "Sita", "marks": 90}
]

for student in students:
    if student["marks"] < 35:
        continue
    print("Certificate Sent for :", student["name"])

=====

if
if-else
if-elif-else
Nested if
match case
for loop
while loop
break
continue

=====
```